

# Colectii cu disciplina de prelucrare dictata de ordinea inserarii elementelor

---

**Stiva:** ultimul venit – primul servit  
(**L**ast **I**n **F**irst **O**ut – **LIFO**)



**Coadă:** primul venit – primul servit  
(**F**irst **I**n **F**irst **O**ut – **FIFO**)

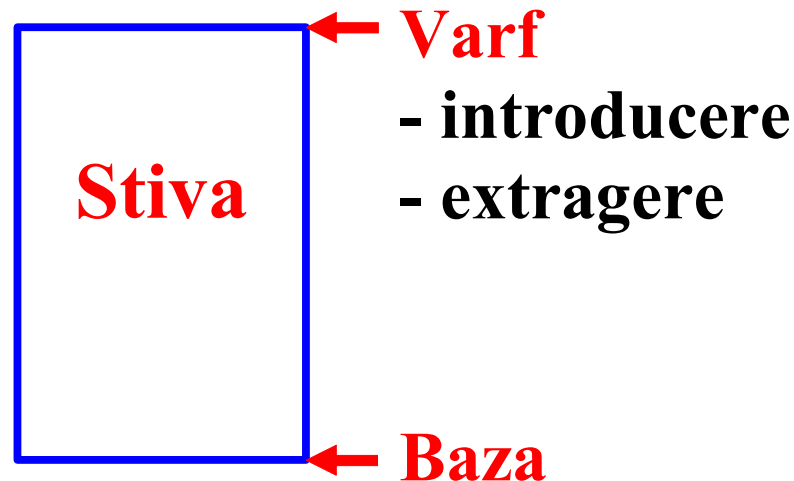


# Colectii cu disciplina de prelucrare dictata de ordinea inserarii elementelor

---

**Stiva:** ultimul venit – primul servit  
(**L**ast **I**n **F**irst **O**ut – **LIFO**)

*Analogii:* tub de medicamente, dosare puse  
teanc in ordinea primirii si parcurse la finalul  
zilei in ordinea in care sunt asezate etc.

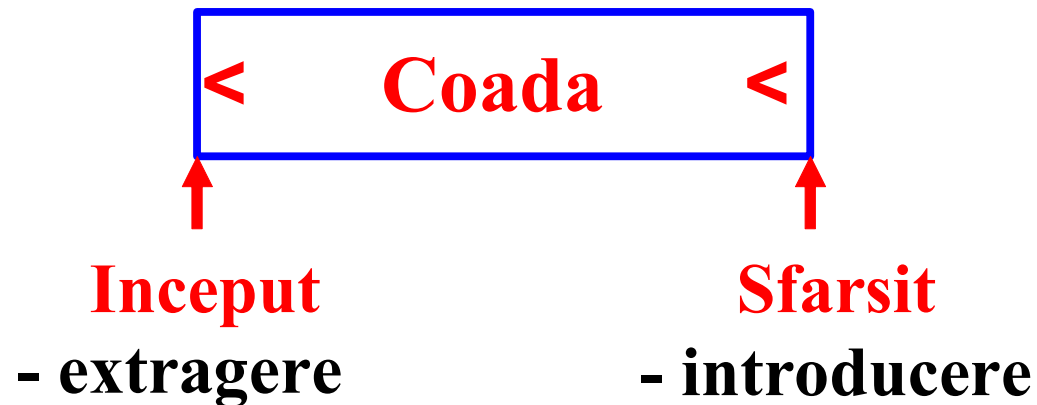


# Colectii cu disciplina de prelucrare dictata de ordinea inserarii elementelor

---

**Cooda:** primul venit – primul servit  
(**F**irst **I**n **F**irst **O**ut – **FIFO**)

*Analogii:* cereri rezolvate in ordinea inregistrarii



# Operatii de baza

---

**Stiva: (Last In First Out – LIFO):**

- Initializare stiva (InitS)
- Adauga element in stiva (Push)
- Extrage element din stiva (Pop)
- Obtine elementul din varful stivei – fara a modifica continutul acesteia (Top)
- Elibereaza spatiul ocupat de intreaga structura (DistrS)

# Operatii de baza

---

## **Cooda: (Last In First Out – LIFO)**

- Initializare coada (InitQ)
- Adauga element in coada (IntrQ)
- Extrage element din coada (ExtrQ)
- Obtine primul element din coada – fara a modifica continutul acesteia (PrimQ)
- Elibereaza spatiul ocupat de intreaga structura (DistrQ)

## Alte operatii

---

- Test structura vida
- Numar elemente
- Dimensiune element din structura
- Transforma structura in structura vida
- Muta primul element din structura sursa in structura destinatie
- Transfera continutul structurii sursa in structura destinatie

# Mutarea unui element

**Coda:** `int MutaQ(void* ad, void* as);`

| Destinatie             | Sursa                    |
|------------------------|--------------------------|
| < 1, 2, 3 <            | < <b>20</b> , 13, 8, 5 < |
| ⇓                      |                          |
| < 1, 2, 3, <b>20</b> < | < 13, 8, 5 <             |

**Stiva:** `int MutaS(void* ad, void* as);`

| Destinatie          | Sursa                 |
|---------------------|-----------------------|
| 1, 2, 3 :           | 20, 13, 8, <b>5</b> : |
| ⇓                   |                       |
| 1, 2, 3, <b>5</b> : | 20, 13, 8 :           |

Echivalenta cu **extragere** urmata de **introducere**.

# Transfer continuut Coada / Stiva

## Concatenare cozi

```
int ConcatQ(void* ad, void* as);
```

**Destinatie**

**Sursa**

< 1, 2, 3 <

< 13, 8, 5 <



< 1, 2, 3, 13, 8, 5 <

<<

## Rastoarna stiva **sursa** in stiva destinatie

```
int Rastoarna(void* ad, void* as);
```

**Destinatie**

**Sursa**

| 11, 22 :

| 21, 22, 23 :



| 11, 22, 23, 22, 21 :

|:

Pot fi realizate prin **mutari repetate**.



# Suprapunere Stive

```
int Suprapune (void* ad, void* as) ;
```

| Destinatie           | Sursa        |
|----------------------|--------------|
| 11, 22 :             | 21, 22, 23 : |
| ⇓                    |              |
| 11, 22, 21, 22, 23 : | :            |

Poate fi realizata prin **mutari repetate**, folosind o **stiva auxiliara**:

```
{ initializeaza stiva aux;  
  cat timp sursa nevida  
  { muta element din sursa in aux; }  
  cat timp aux nevida  
  { muta element din aux in destinatie; }  
  distruge stiva aux;  
}
```

# Inversare Stiva

---

```
int InvS(void* a);
```

| 1, 2, 3, 4, 5 :

⇒

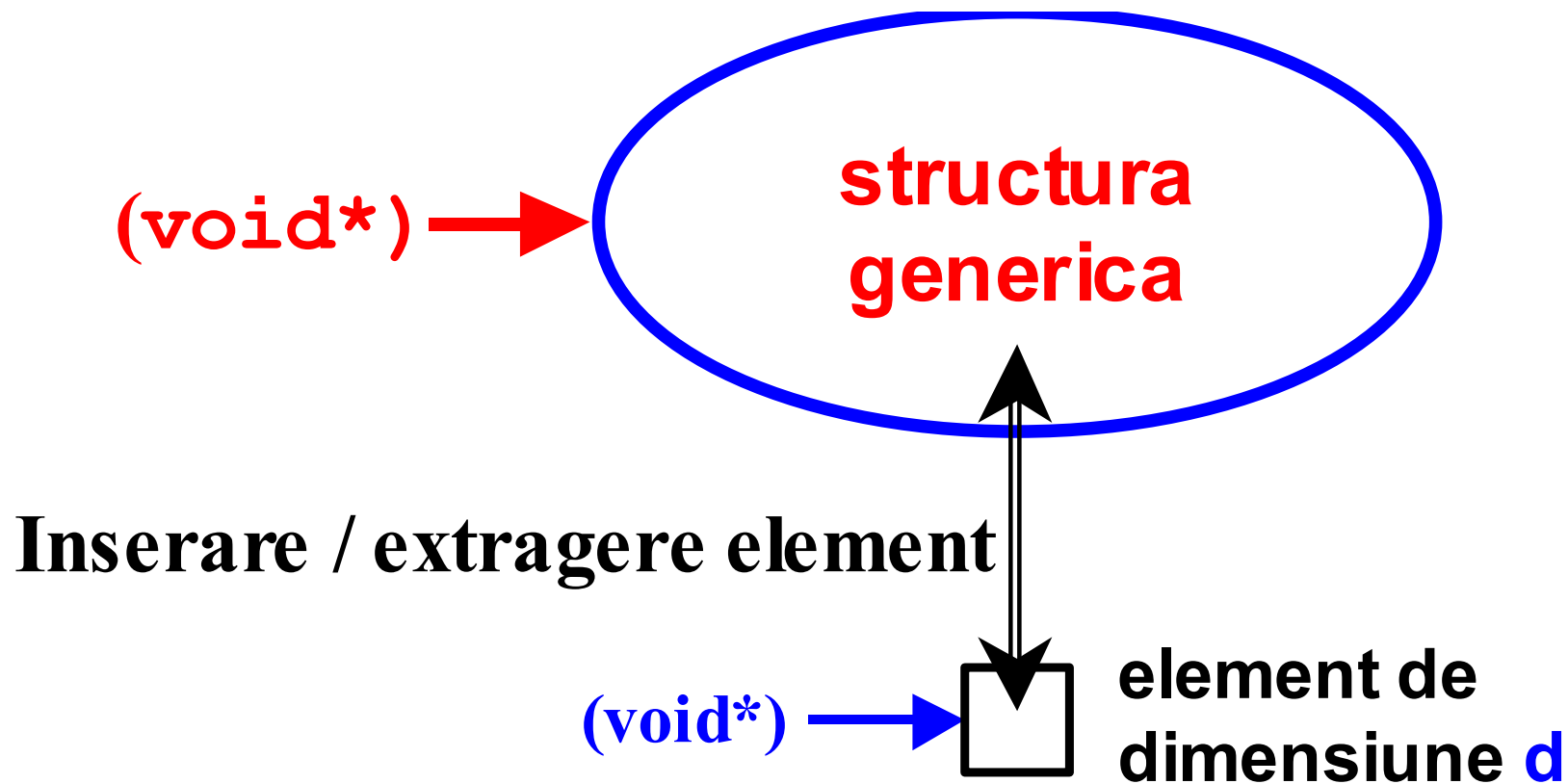
| 5, 4, 3, 2, 1 :

Poate fi realizata prin **mutari repetate**, folosind  
o **coada auxiliara**:

```
{ initializeaza coada;  
  cat timp stiva nevida  
  { extrage element din stiva;  
    introduce elementul in coada;  
  }  
  cat timp coada nevida  
  { extrage element din coada;  
    introduce elementul in stiva;  
  }  
  distruge coada;  
}
```

# Coada si Stiva generice

---



# Coada si Stiva generice - operatii standard

---

**Initializare** structura cu elemente de dimensiune **d**:

```
void* InitQ(size_t d, ...);
```

```
void* InitS(size_t d, ...);
```

**Rezultat:** NULL sau adresa structura

**Inserare:**

```
int IntrQ(void* a, void* ae);
```

```
int Push(void* a, void* ae);
```

**Rezultat:** 1 – succes / 0 - esec

# Coada si Stiva generice - operatii standard

---

## Extragere

```
int ExtrQ(void* a, void* ae);  
int Pop(void* a, void* ae);
```

Rezultat: 1 – succes / 0 - esec

## Eliberarea intregului spatiu ocupat de structura:

```
void DistrQ(void** aa);  
void DistrS(void** aa);
```

## Test structura vida:

```
int VidaQ(void* a);  
int VidaS(void* a);
```

# Coada si Stiva generice - operatii standard

---

## Copiere element accesibil:

```
int PrimQ(void* a, void* ae);  
int Top(void* a, void* ae);
```

## Numar elemente:

```
size_t NrEQ(void* a);  
size_t NrES(void* a);
```

## Dimensiune elemente:

```
size_t DimEQ(void* a);  
size_t DimES(void* a);
```

# Cooda si Stiva generice – alte operatii

---

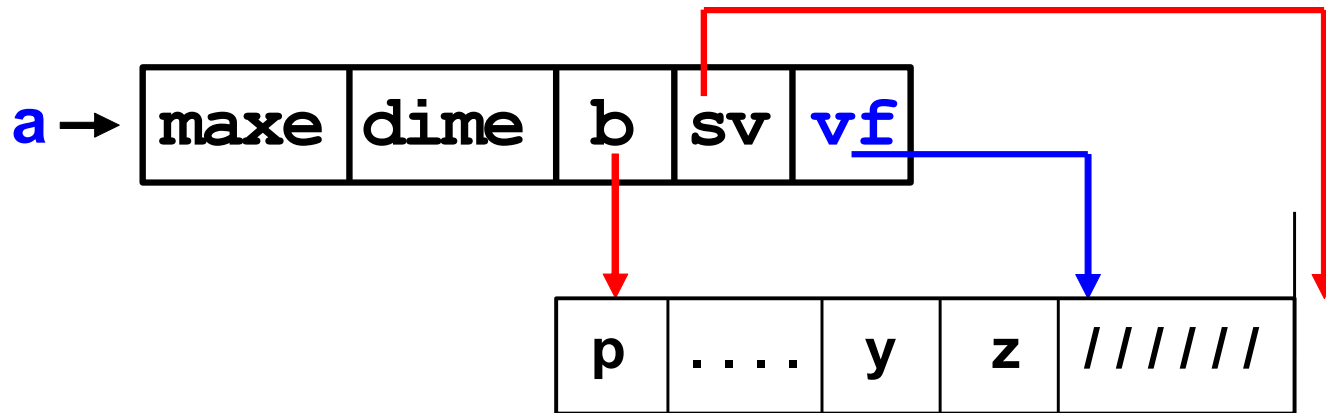
## Transformare in structura vida:

```
void ResetQ(void* a);
```

```
void ResetS(void* a);
```

# Implementare **stiva** generica – A [1]

A. Folosind un **vector** de elemente de dimensiune **d**



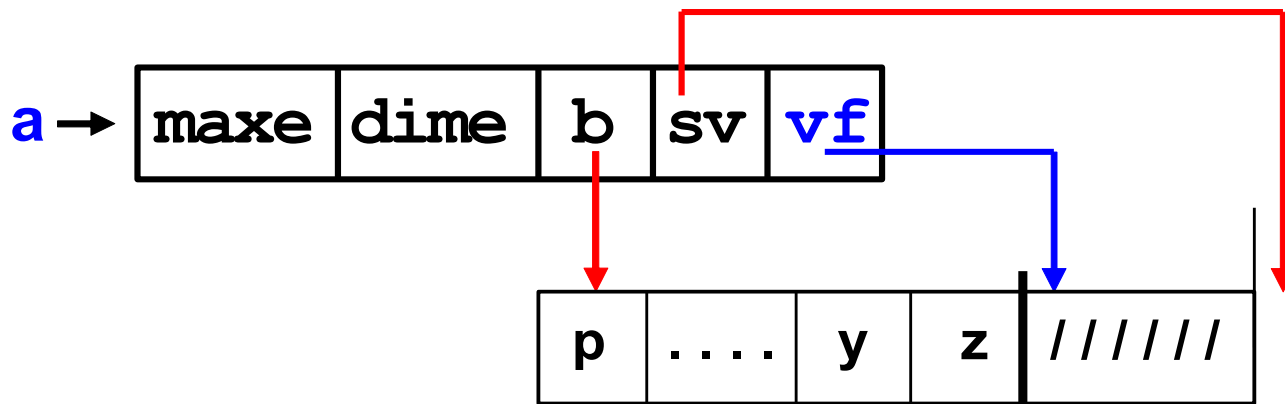
```
typedef struct
{ size_t maxe, dime; /* nr.maxim elemente,
                      dimensiune element */
  char *b, *sv, *vf; /* baza, sfarsit vector,
                      varf stiva */
} TStiva, *ASt;
```

Stiva vida: **`a->vf == a->b`**



# Implementare **stiva** generica – A [2]

```
/*- macrodefinitii utile -*/  
/*- acces la campuri -*/  
#define DIME(a) (((Ast)(a))->dime)  
#define BS(a) (((Ast)(a))->b)  
#define SV(a) (((Ast)(a))->sv)  
#define VF(a) (((Ast)(a))->vf)  
/*- teste -*/  
#define DIMDIF(s,d) (DIME(s) != DIME(d))  
#define VIDA(a) (VF(a) == BS(a))  
#define PLINA(a) (VF(a) == SV(a))
```



# Implementare **stiva** generica – A [3]

## Initializare

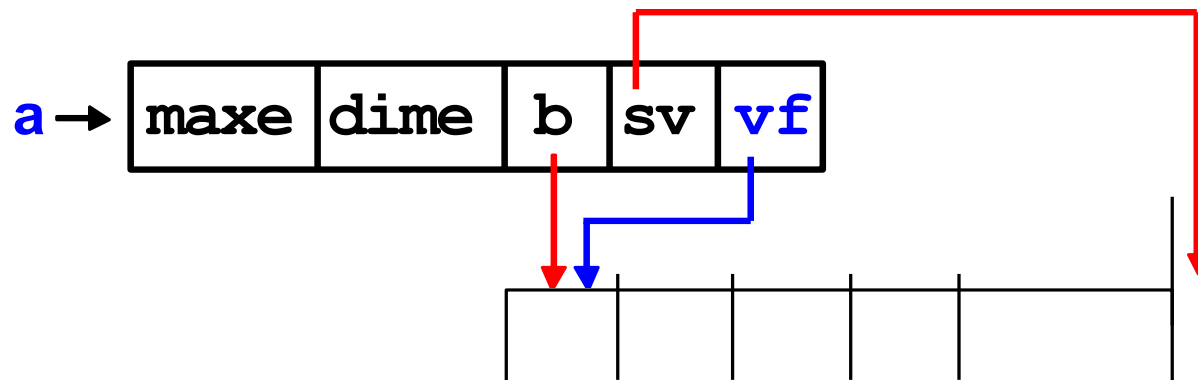
```
a = InitS(sizeof(tip_element), maxim);
```

In cazul unei initializari reusite

```
a->maxe == maxim
```

```
a->dime == sizeof(tip_element)
```

```
a->sv == a->b + a->maxe * a->dime
```



# Implementare **stiva** generica – A [4]

```
void* InitS(size_t d,...)
```

```
{
```

```
    ASt a;
```

```
    va_list ap;
```

```
    a = (ASt)malloc(sizeof (TStiva));
```

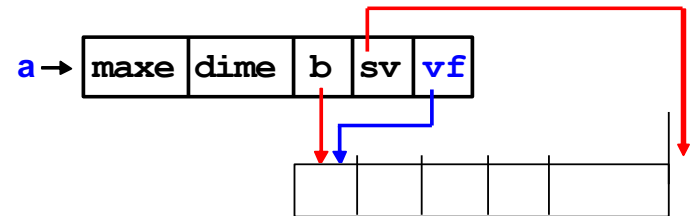
```
    if (!a) return NULL;
```

```
    va_start(ap,d);
```

```
    a->maxe = va_arg(ap,size_t);
```

```
    va_end(ap);
```

```
    a->dime = d;
```



# Implementare **stiva** generica – A [5]

```
/* continuare InitS(size_t d,...)*/
```

```
/* aloca spatiu pentru vector si test esec */
```

```
a->b = (char*)calloc(a->maxe, d);
```

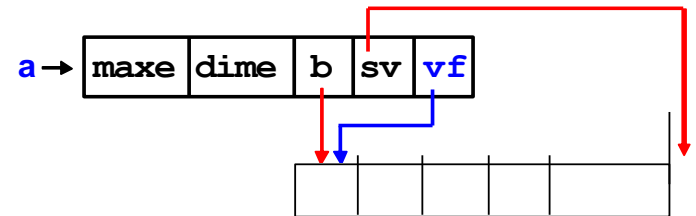
```
if (!a->b)
```

```
{
```

```
    free(a);
```

```
    return NULL;
```

```
}
```



```
/* actualizeaza adrese din descriptor */
```

```
a->vf = a->b;
```

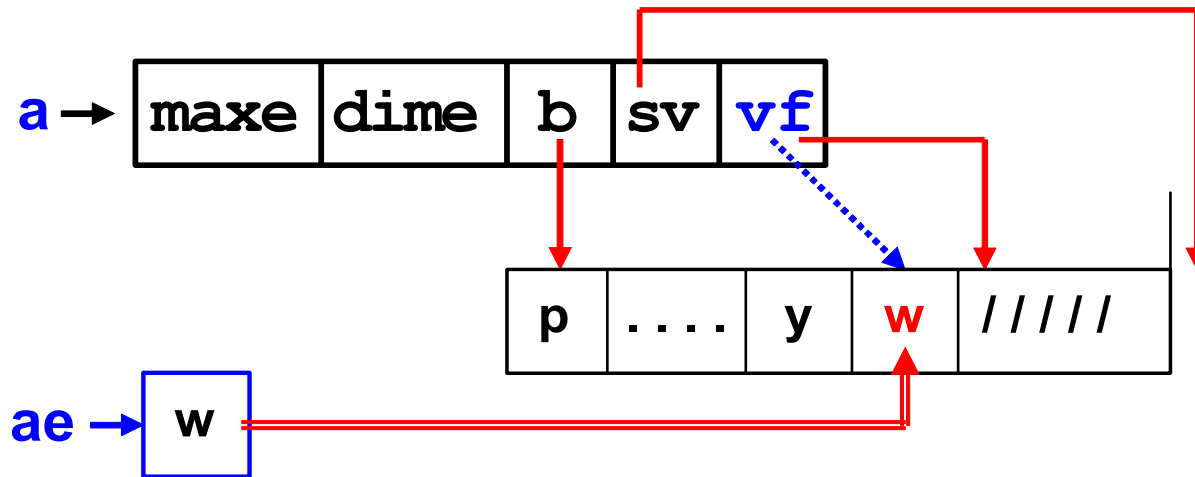
```
a->sv = a->b + d * a->maxe;
```

```
return (void *)a;
```

```
}
```

# Implementare **stiva** generica – A [6]

## Inserare



```
int Push(void* a, void* ae)
```

```
{
```

```
    if (PLINA(a)) return 0;
```

```
    memcpy (VF(a), ae, DIME(a));
```

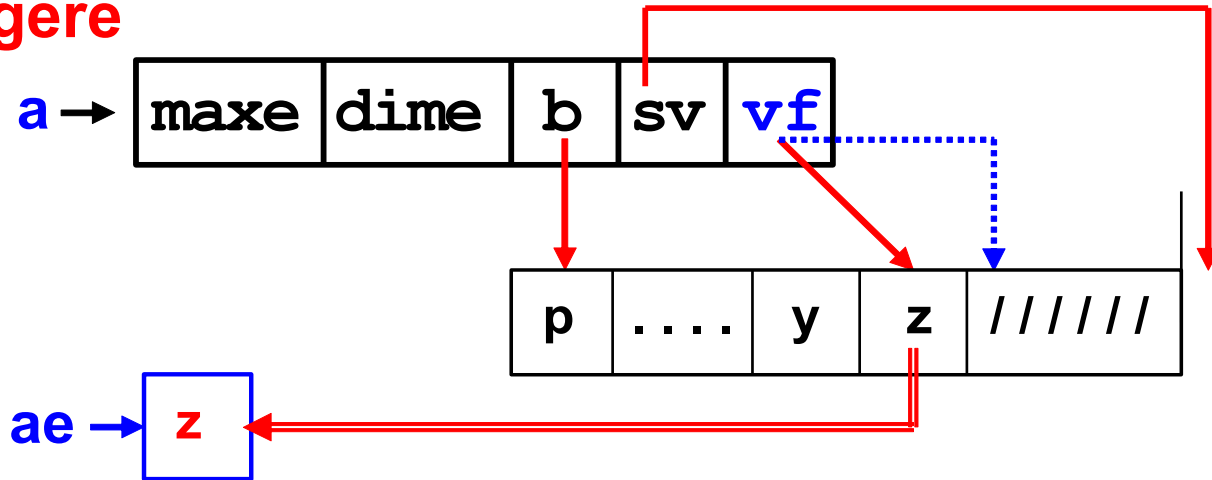
```
    VF(a) += DIME(a);
```

```
    return 1;
```

```
}
```

# Implementare **stiva** generica – A [7]

## Extragere



```
int Pop(void* a, void* ae)
```

```
{
```

```
    if (VIDA(a)) return 0;
```

```
    VF(a) -= DIME(a);
```

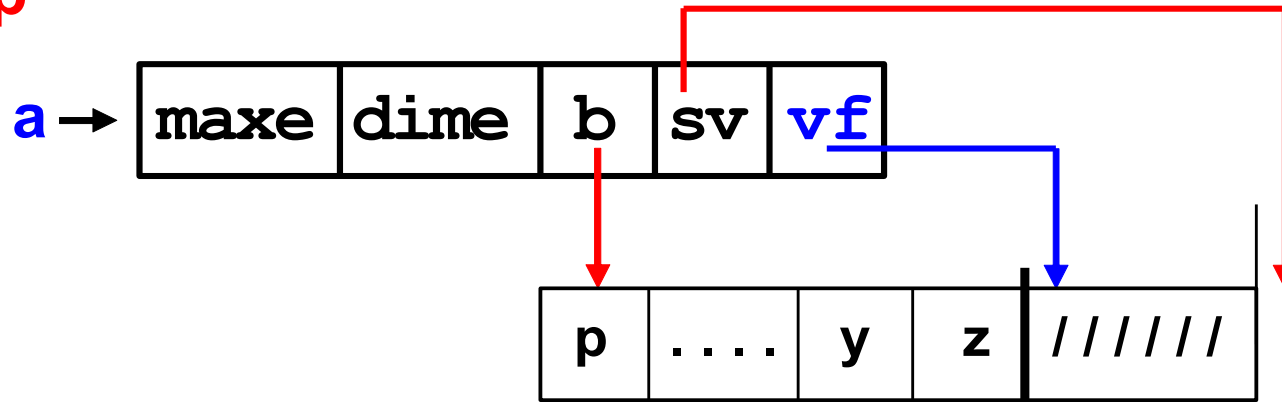
```
    memcpy(ae, VF(a), DIME(a));
```

```
    return 1;
```

```
}
```

# Implementare **stiva** generica – A [8]

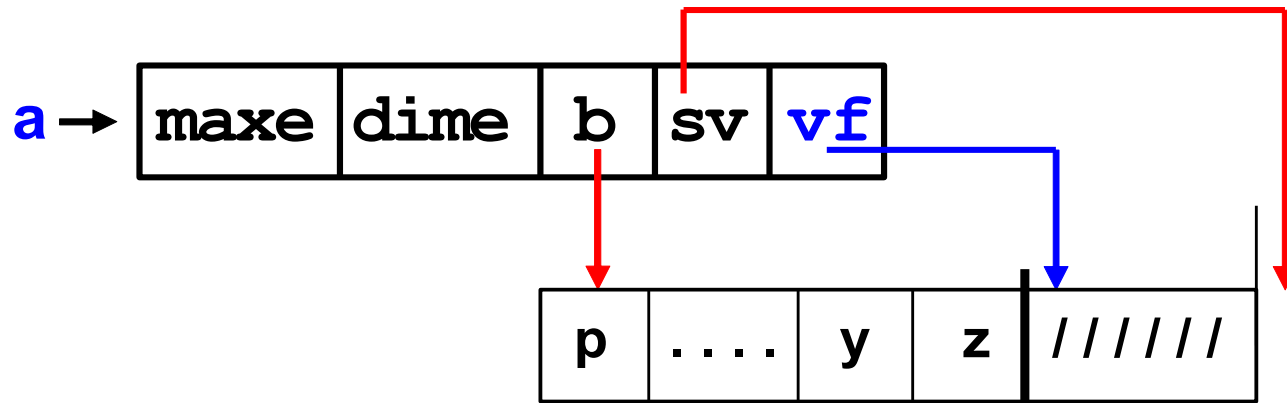
**Top**



```
int Top(void* a, void* ae)
{
    if (VIDA(a)) return 0;
    memcpy (ae, VF(a) - DIME(a), DIME(a));
    return 1;
}
```

# Implementare **stiva** generica – A [9]

## Numar elemente



```
int NrES(void* a)
```

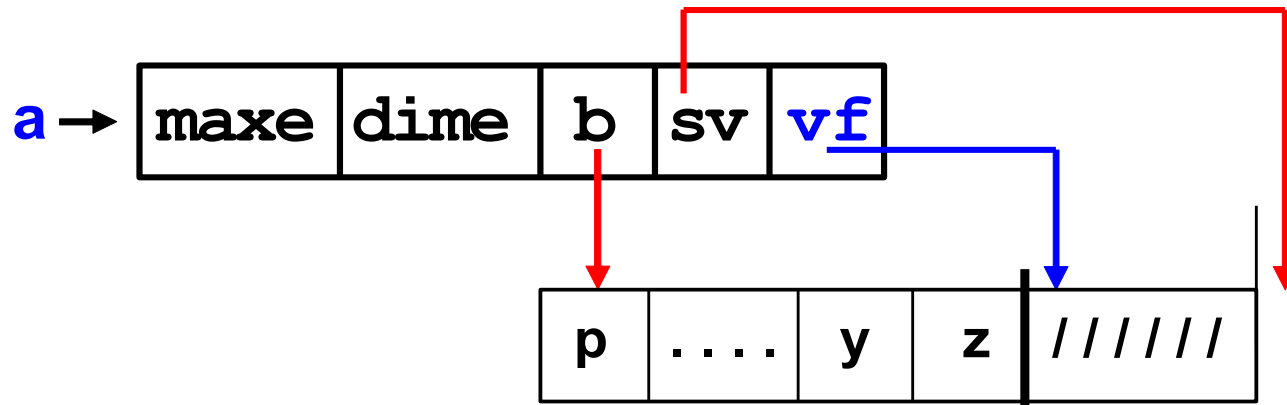
*intoarce numarul de elemente din stiva*

$((VF(a) - BS(a)) / DIME(a))$



# Implementare **stiva** generica – A [10]

## Reset



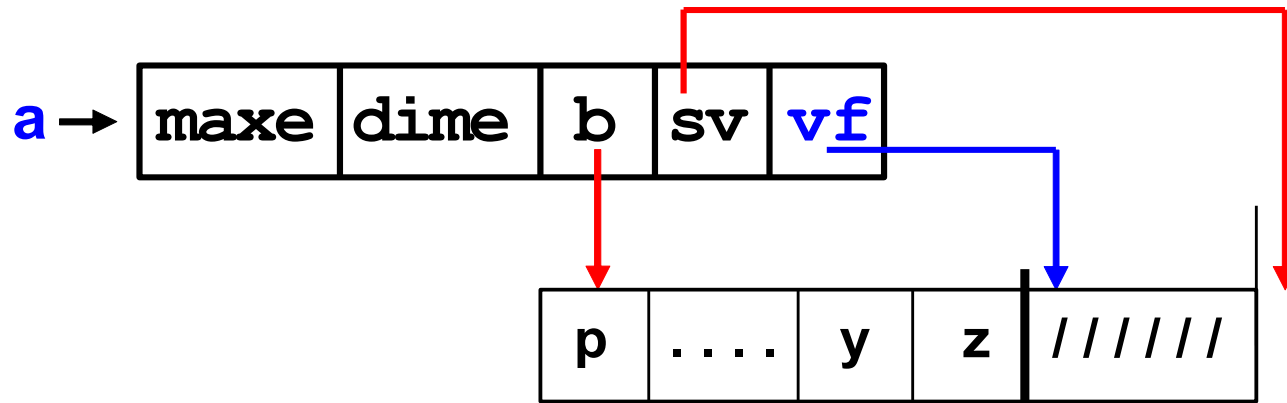
```
void ResetS(void* a)
```

```
/* transforma stiva in stiva vida*/
```

```
seteaza varful stivei la baza acesteia
```

# Implementare **stiva** generica – A [11]

## Distrugere stiva



```
void DistrS(void** a)
```

```
/* elibereaza intregul spatiu ocupat de stiva*/
```

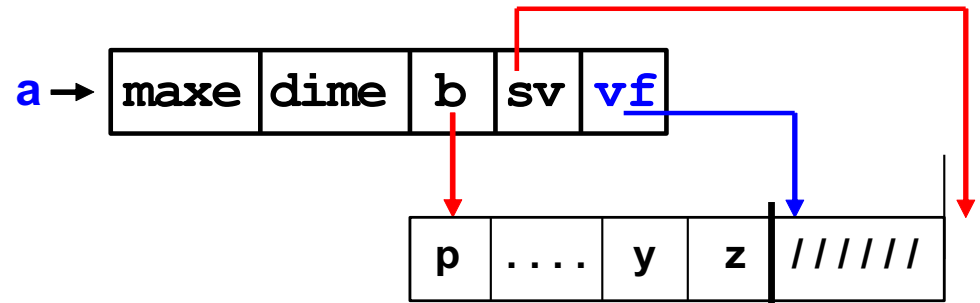
```
    elibereaza spatial ocupat de vector
```

```
    elibereaza spatial ocupat de descriptorul structura
```

```
    marcheaza stiva ca vida
```

# Implementare **stiva** generica – A [12]

```
void DistrS(void **aa)
{
    free(BS(*aa));
    free(*aa);
    *aa = NULL;
}
```



# Implementare **stiva** generica – A [13]

---

```
int MutaS(void *ad, void *as)
{
    if(DIMDIF(as,ad)) return -1;
    if(VIDA(as)) return 0;
    if(PLINA(ad)) return -2;

    /* mutare element */
    VF(as) -= DIME(as);
    memcpy(VF(ad), VF(as), DIME(as));
    VF(ad) += DIME(ad);
    return 1;
}
```

# Implementare **stiva** generica – A [14]

```
int Suprapune(void *ad, void *as)
{
    size_t n = VF(as) - BS(as);
    if(DIMDIF(as,ad)) return -1;
    if(VIDA(as)) return 0;
    if(VF(ad) + n > SV(ad)) return -2;

    /* transfer elemente */
    memcpy(VF(ad), BS(as), n);
    VF(ad) += n;
    VF(as) = BS(as);
    return 1;
}
```

**Destinatie**

**Sursa**

| 11, 22 :

| **21, 22, 23 :**



| 11, 22, **21, 22, 23 :**

|:

# Implementare **stiva** generica – A [15]

```
int Rastoarna (void *ad, void *as)
```

```
{
```

```
    size_t n = VF(as) - BS(as);
```

```
    if(DIMDIF(as,ad)) return -1;
```

```
    if(VIDA(as)) return 0;
```

```
    if(VF(ad) + n > SV(ad)) return -2;
```

```
    /* muta din varf sursa in varf destinatie */
```

```
    while(VF(as) > BS(as))
```

```
    {
```

```
        VF(as) -= DIME(as);
```

```
        memcpy(VF(ad), VF(as), DIME(as));
```

```
        VF(ad) += DIME(as);
```

```
    }
```

```
    return 1;
```

```
}
```

Destinatie

Sursa

| 11, 22 :

| 21, 22, 23 :



| 11, 22, 23, 22, 21 :

|:

# Implementare **stiva** generica – A [16]

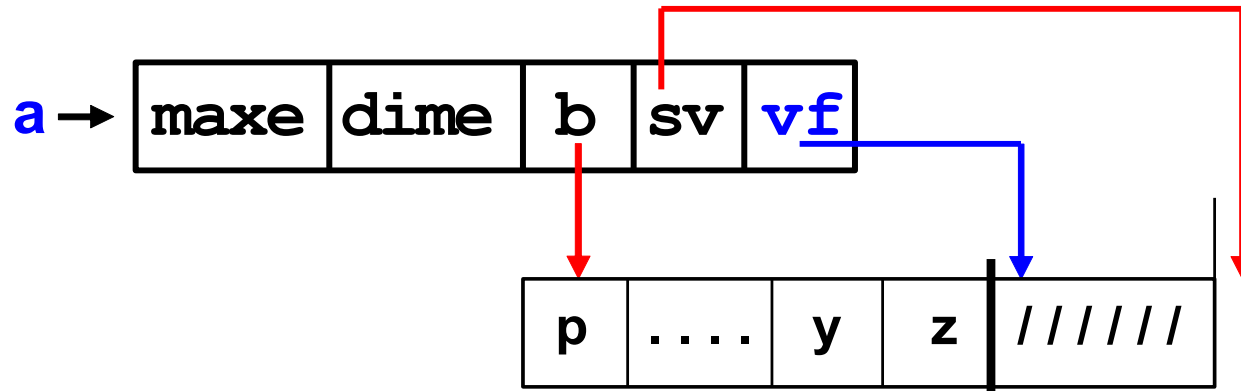
---

```
int InvS(void *a)
{
    char *temp = VF(a), *p, *q;
    if(temp >= SV(a)) temp = (char *)malloc(DIME(a));
    if(!temp) return 0;
    /* inverseaza perechi de elemente; p, q */
    for(p = BS(a), q = VF(a)-DIME(a); p < q;
        p += DIME(a), q -= DIME(a))
    {
        memcpy(temp, p, DIME(a));
        memcpy(p, q, DIME(a));
        memcpy(q, temp, DIME(a));
    }

    if(temp != VF(a)) free(temp);
    return 1;
}
```

# Implementare **stiva** generica – A [12]

---

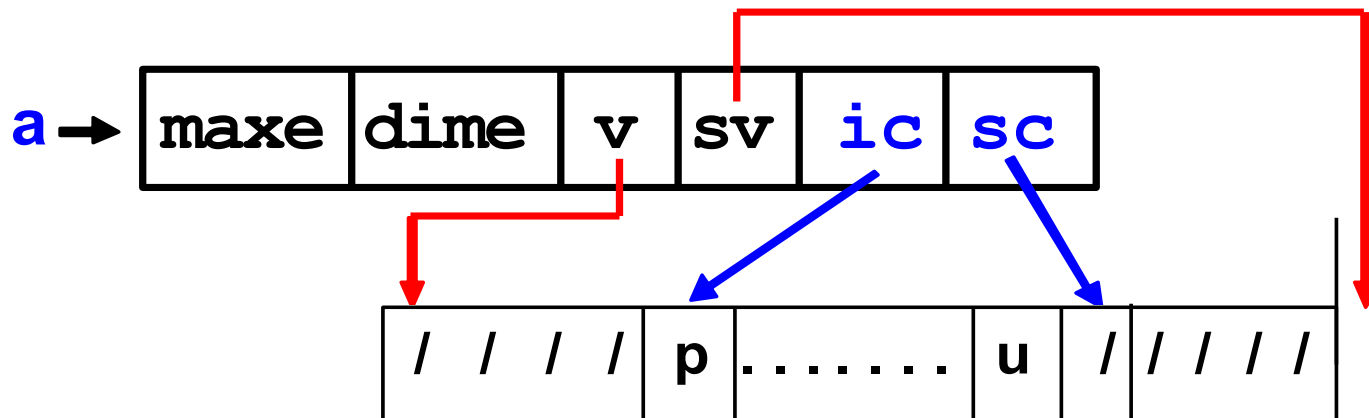




# Implementare **coada** generica – A [1]

A. Folosind un **vector** prelucrat circular, cu santinela

```
typedef struct
{
    size_t maxe, dime;    /* nr.maxim elemente,
                           dimensiune element */
    char *v, *sv,         /* inceput, sfarsit vector */
        *ic, *sc;         /* inceput, sfarsit coada */
} TCoadaV, *AQV;
```

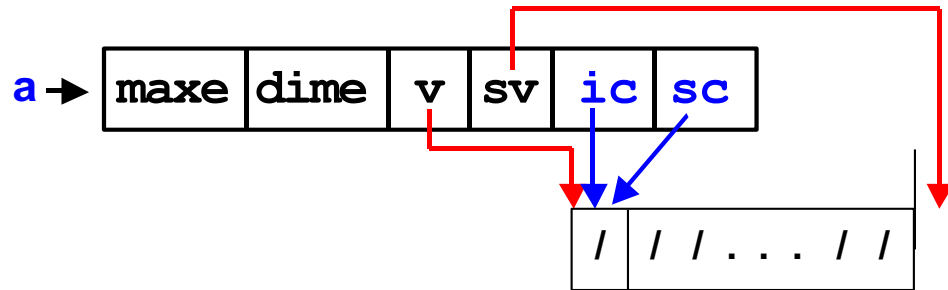


# Implementare coada generica – A [1]

**Coadă vida**

maxe = 6

v, ic, sc



**Inserare 2, 3, 4**

v, ic



sc



**3 eliminari**

v



ic, sc



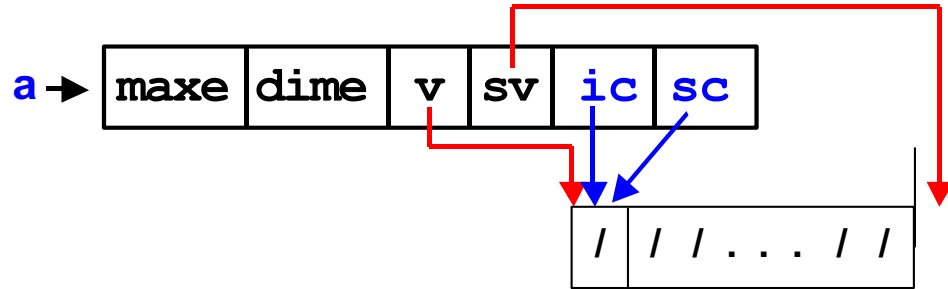
**Coadă vida**

# Implementare coada generica – A [1]

**Coadă vida**

maxe = 6

v, ic, sc



**Inserare 2, 3, 4**

v, ic



sc



**Inserare 5, 6**

v, ic



sc



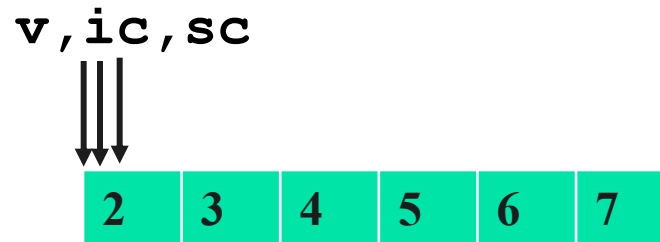
# Implementare **coada** generica – A [1]

---

`maxe = 6`



**Inserare 7**



Avans circular `ic, sc` → coada circulara

# Implementare **coada** generica – A [1]

---

maxe = 6

v, ic, sc



Coadă plină

Extragere doua elemente

v, sc



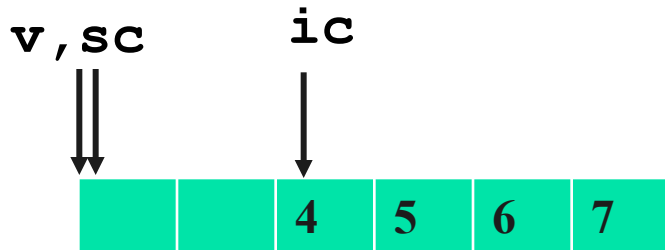
ic



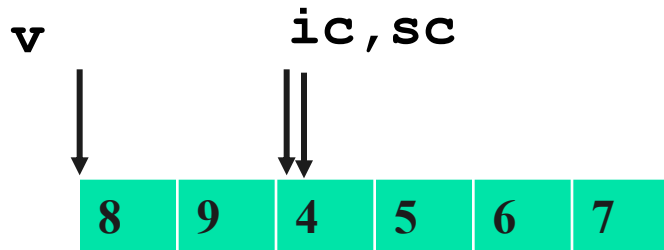
# Implementare **coada** generica – A [1]

---

maxe = 6



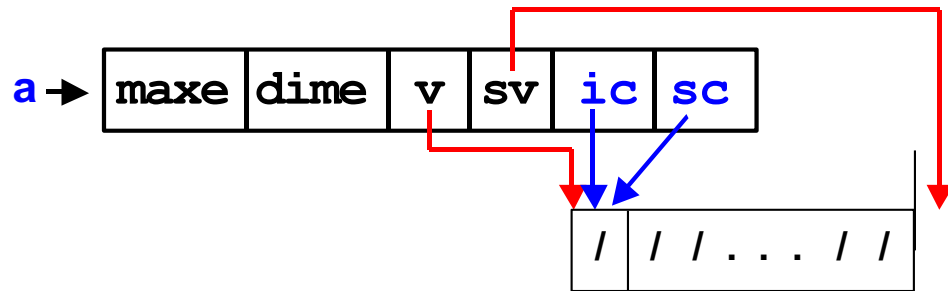
Inserare: 8, 9



**Coadă plină**

# Implementare coada generica – A [1]

Coada cu maxe  
elemente plus un  
element suplimentar  
= santinela



Coada vida: `maxe = 6` (vector alocat cu `maxe+1` elemente)

`v, ic, sc`



`ic == sc`

Inserare: 1, 2, 3

`v, ic`

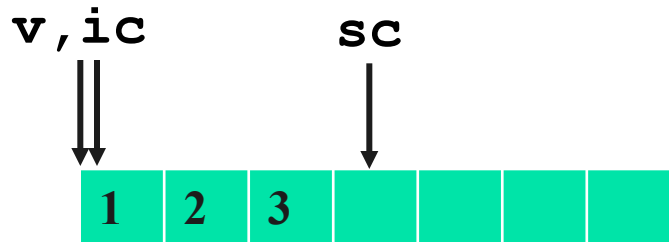


`sc`



# Implementare **coada** generica – A [1]

---



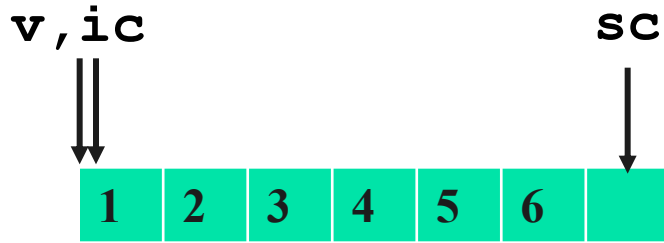
**Inserare: 4, 5, 6**



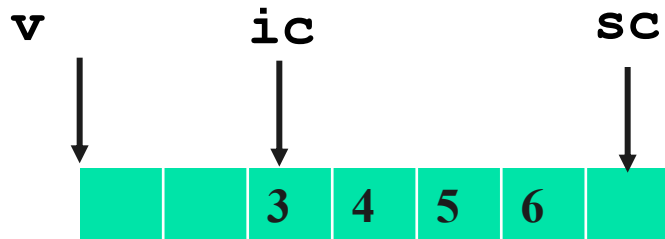
**Coda plina**



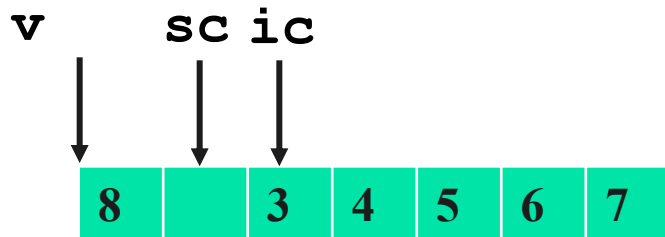
# Implementare **coada** generica – A [1]



**2 eliminari**



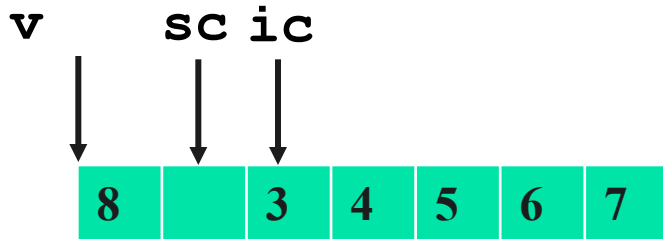
**Inserare: 7, 8**



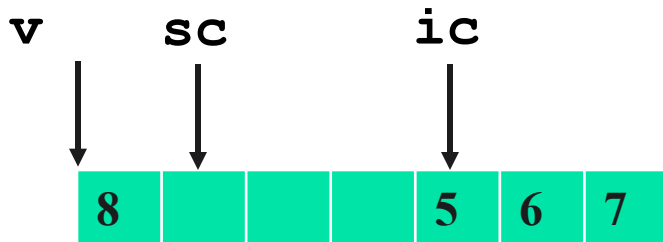
**Coadă plină**

# Implementare **coada** generica – A [1]

---



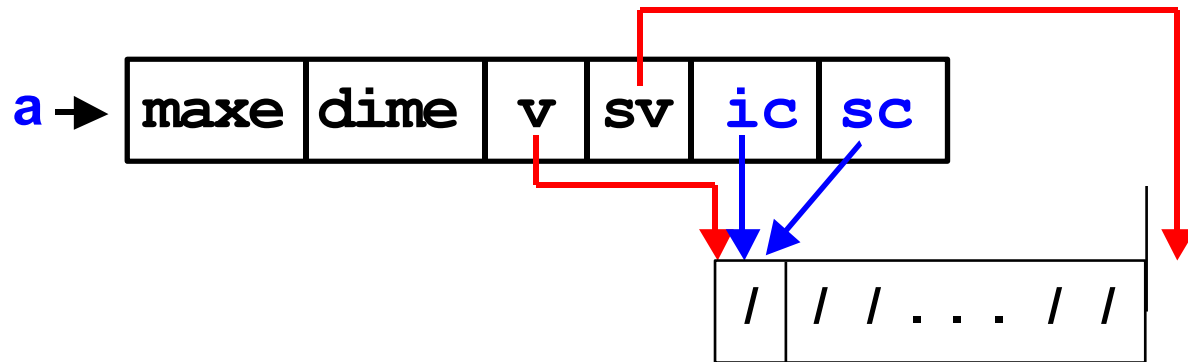
**2 eliminari**



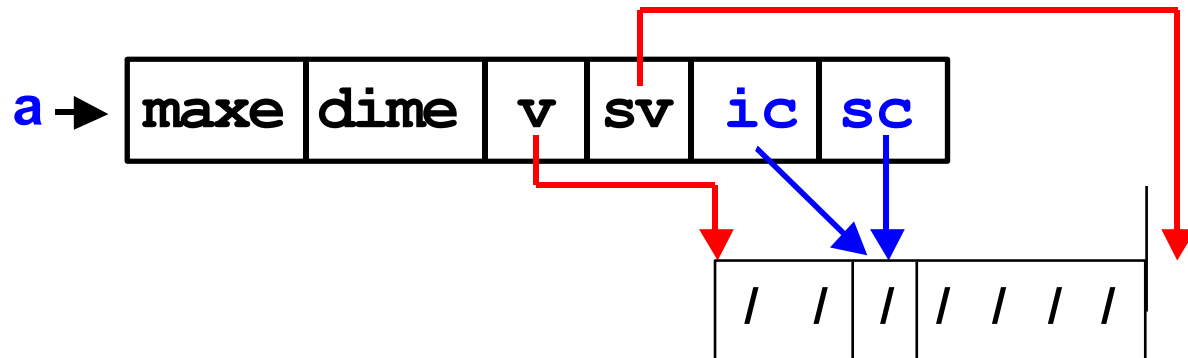
# Implementare coada generica – A [2]

Cooda vida:  $a \rightarrow ic == a \rightarrow sc$

- dupa initializare

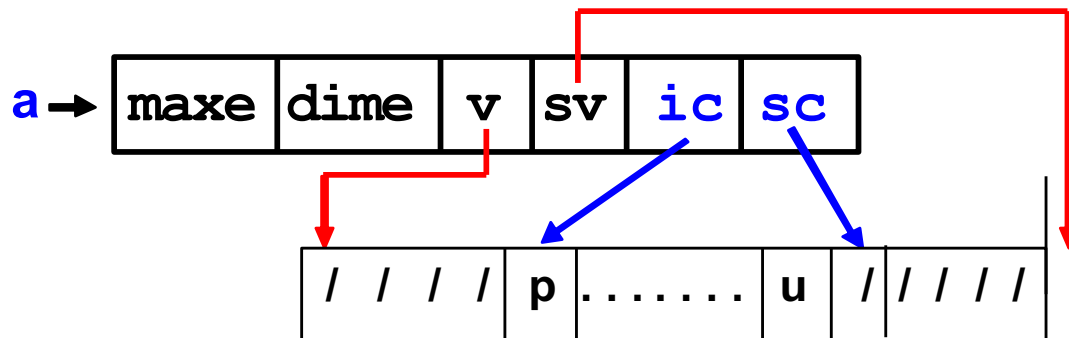


- dupa numar egal de inserari si eliminari



# Implementare **coada** generica – A [3]

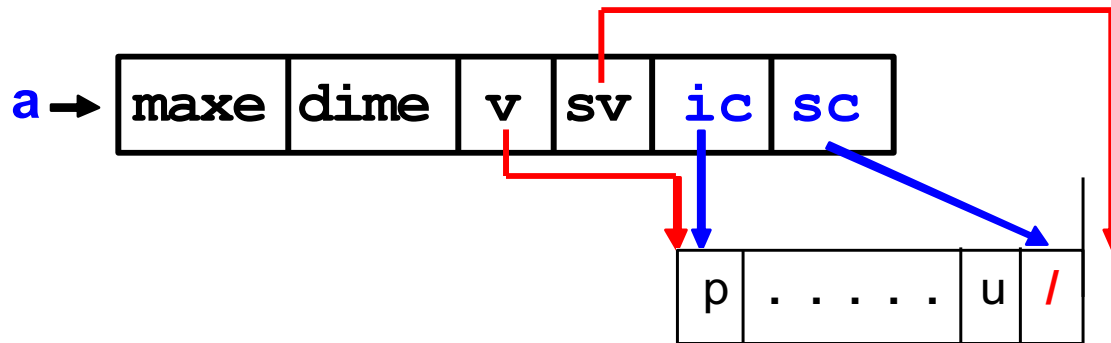
```
/*- macrodefinitii utile -*/  
/*- acces la campuri -*/  
#define V(a) ((AQV)(a))->v  
#define SV(a) ((AQV)(a))->sv  
#define IC(a) ((AQV)(a))->ic  
#define SC(a) ((AQV)(a))->sc  
#define DIME(a) ((AQV)(a))->dime)  
/*- teste si operatii simple -*/  
#define DIMDIF(s,d) (DIME(s) != DIME(d))  
#define VIDA(a) (IC(a) == SC(a))
```



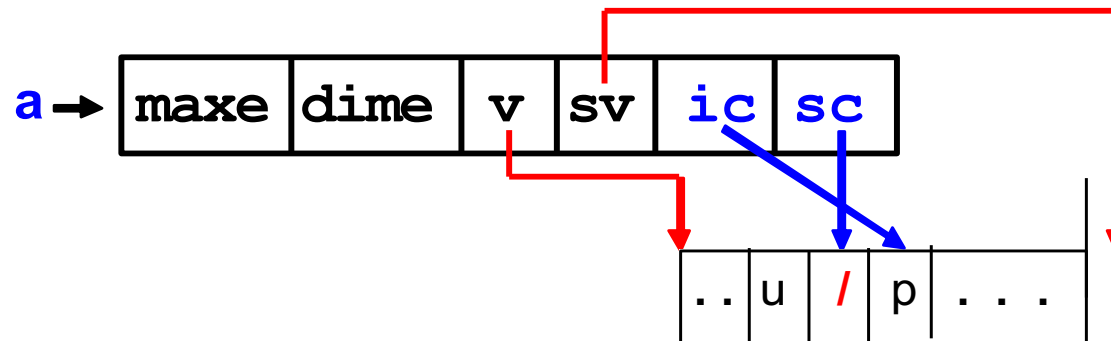
# Implementare coada generica – A [6]

## Coadă plină:

- cazul  $IC(a) == V(a) : SC(a) + DIME(a) == SV(a)$



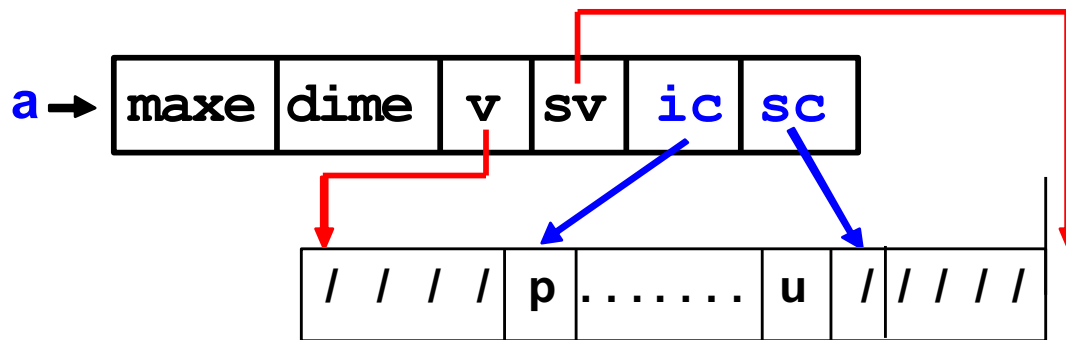
- cazul  $IC(a) > V(a) : SC(a) + DIME(a) == IC(a)$



# Implementare **coada** generica – A [4]

**Avans** (*a*, *p*)

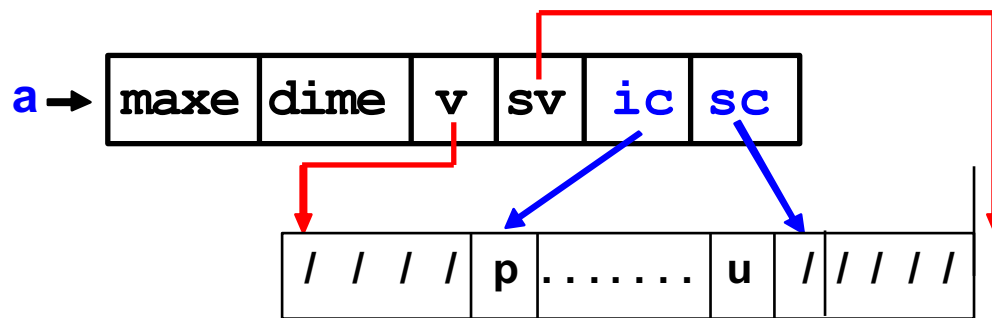
*daca*  $p + DIME(a) < SV(a)$  *atunci*  $p += DIME(a)$   
*altfel*  $p = V(a)$



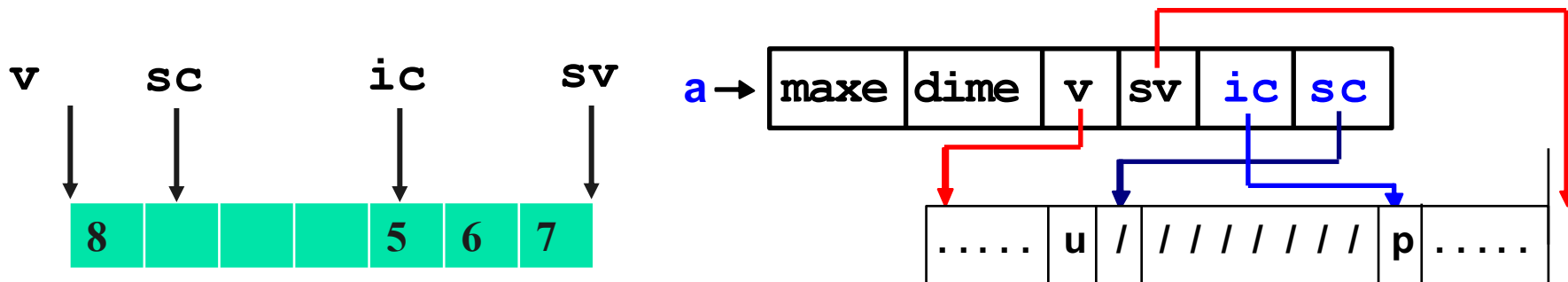
# Implementare coada generica – A [5]

Spatiu ocupat, exprimat in octeti: Ocupat(a)

- cazul  $IC(a) \leq SC(a)$  :  $SC(a) - IC(a)$

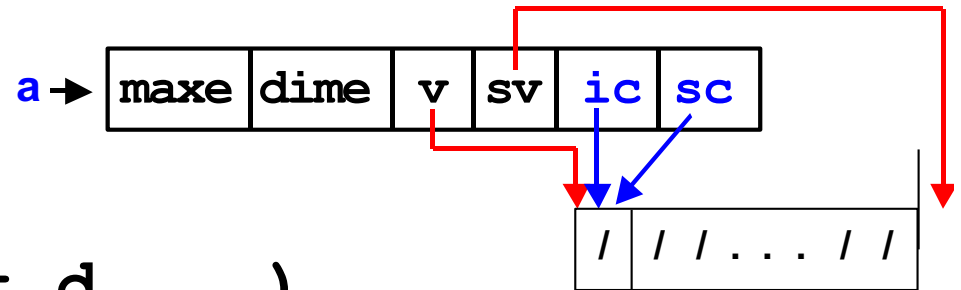


- cazul  $IC(a) > SC(a)$  :  $SC(a) - V(a) + SV(a) - IC(a)$



# Implementare **coada** generica – A [7]

## Initializare



```
void* InitQ(size_t d,...)
{
    A_QV a = (A_QV)malloc(sizeof(TCodaV));
    va_list ap;

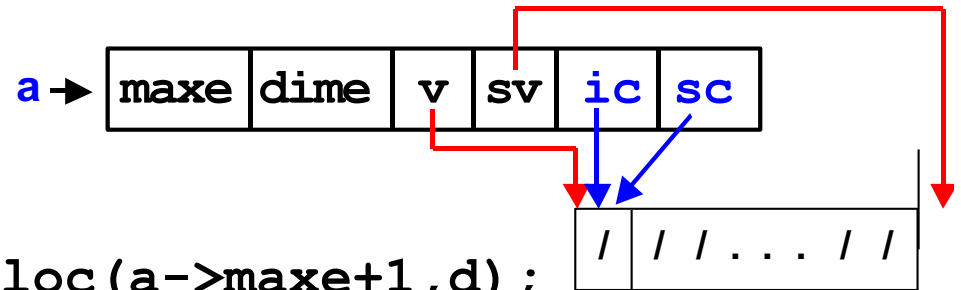
    if (!a) return NULL;
    /* preia param.2 = nr.max.elemente */
    va_start(ap,d);
    a->maxe = va_arg(ap,size_t);
    va_end(ap);

    a->dime = d;
```



# Implementare coada generica – A [8]

## Initializare



```
a->v = (char*)calloc(a->maxe+1,d) ;
```

```
if(!a->v)
```

```
{
```

```
    free(a) ;
```

```
    return NULL;
```

```
}
```

```
/* actualizeaza adrese din descriptor */
```

```
a->sv = a->v + (a->maxe+1) * d;
```

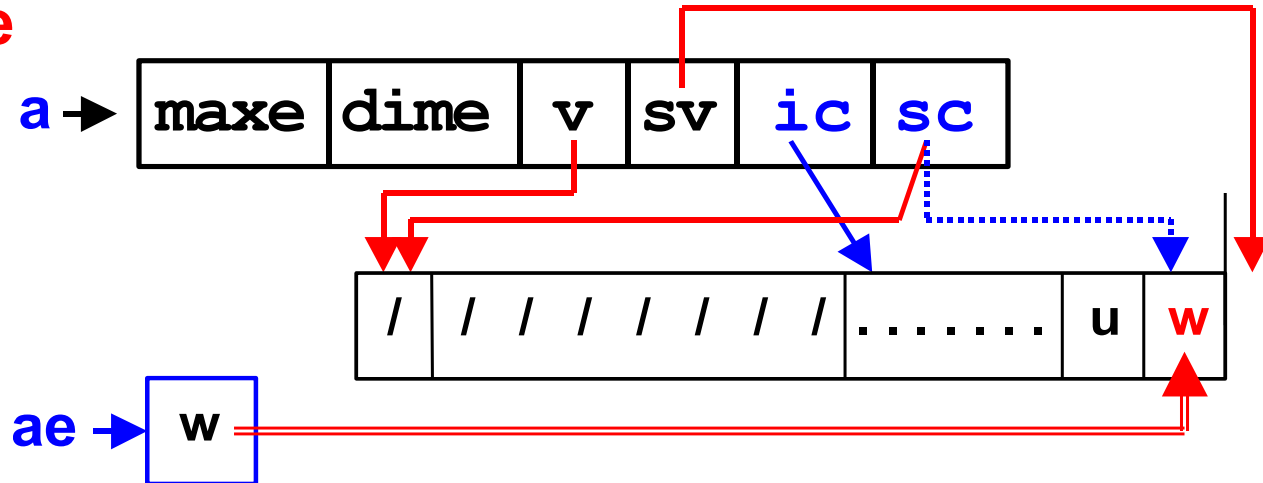
```
a->ic = a->sc = a->v;
```

```
return (void*)a;
```

```
}
```

# Implementare **coada** generica – A [9]

## Inserare



```
int IntrQ(void* a, void* ae)
```

```
{
```

```
    if (PLINA(a)) return 0;
```

```
    memcpy(SC(a), ae, DIME(a));
```

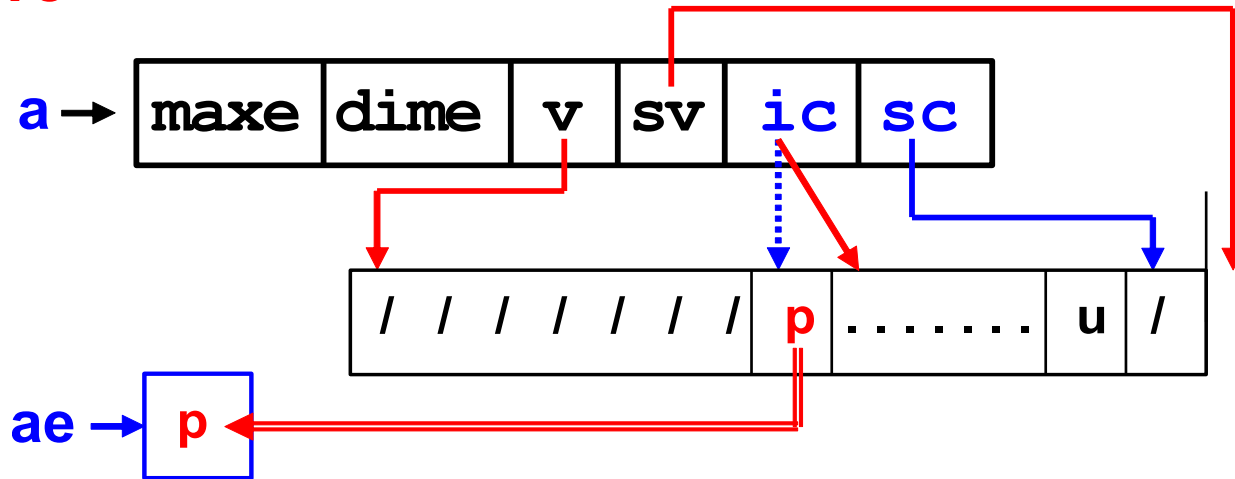
```
    Avans(a, SC(a));
```

```
    return 1;
```

```
}
```

# Implementare coada generica – A [10]

## Extragere

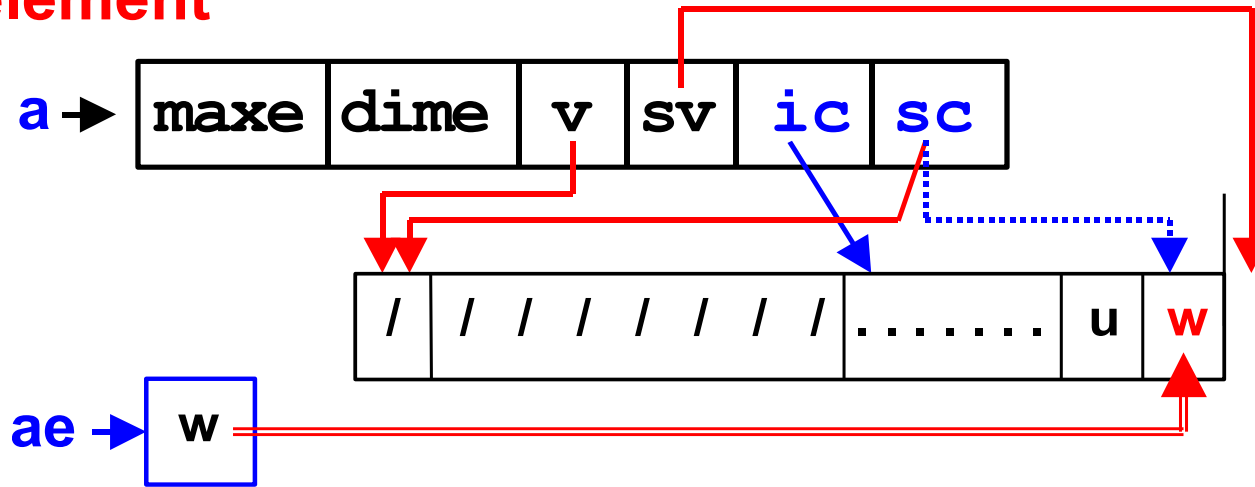


```
int ExtrQ(void* a, void* ae)
```

```
{  
    if(VIDA(a)) return 0;  
    memcpy(ae, IC(a), DIME(a));  
    Avans(a, IC(a));  
    return 1;  
}
```

# Implementare **coada** generica – A [11]

## Primul element



```
int PrimQ(void *a, void *ae)
```

```
{
```

```
    if(VIDA(a)) return 0;
```

```
    memcpy(ae, IC(a), DIME(a));
```

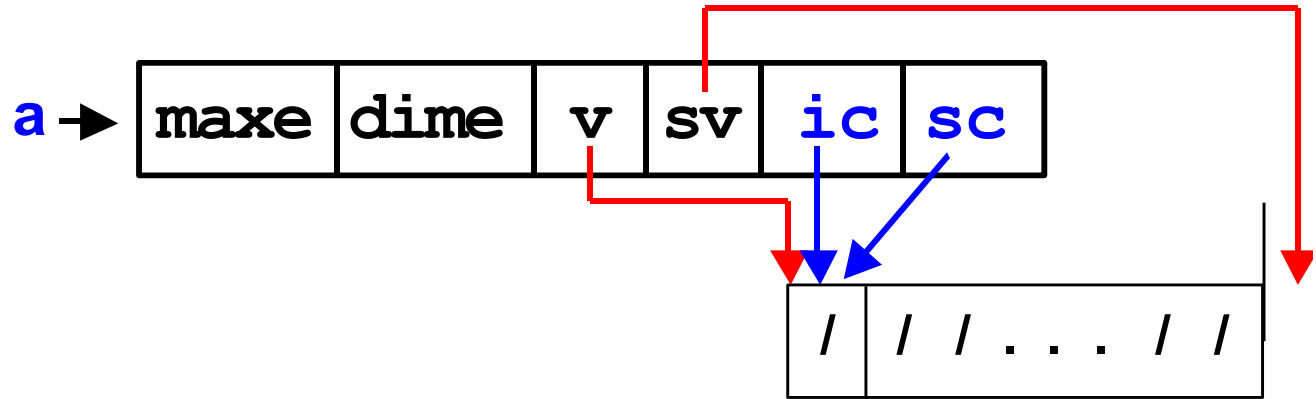
```
    Avans(a, IC(a));
```

```
    return 1;
```

```
}
```

# Implementare coada generica – A [12]

## Numar elemente



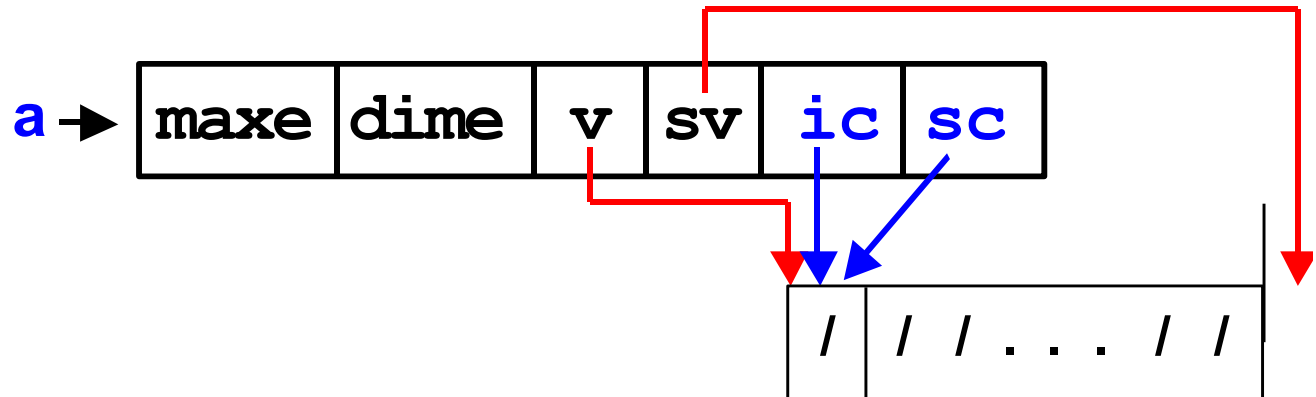
```
size_t NrEQ(void *a)
```

```
/* numarul de elemente din coada */
```

```
    intoarce spatiul ocupat / dimensiune element
```

# Implementare coada generica – A [13]

## Reset



```
void ResetQ(void *a)
```

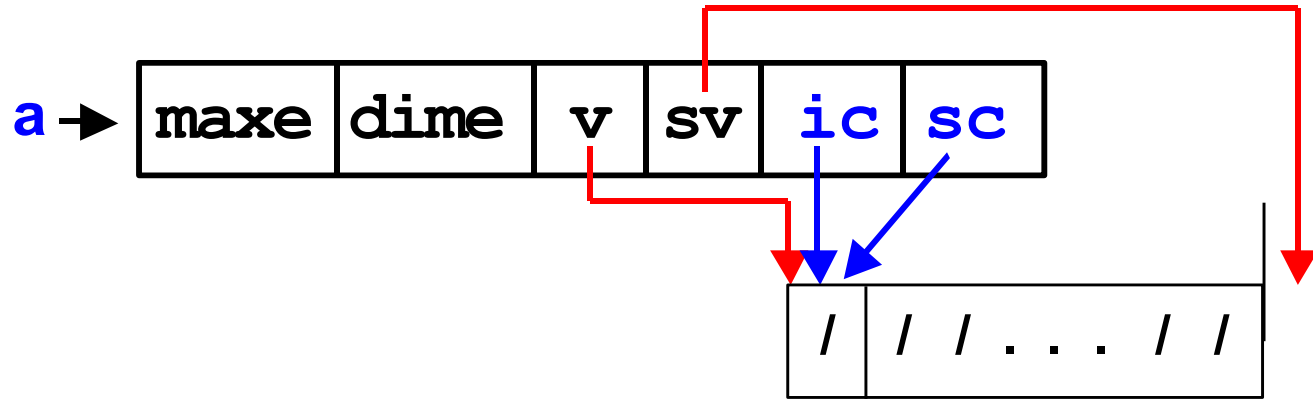
```
/* transforma coada in coada vida */
```

```
seteaza inceputul si sfarsitul cozii la  
inceputul vectorului
```

```
SC(a) = IC(a) = V(a);
```

# Implementare coada generica – A [14]

## Distruge coada



```
void DistrQ(void **a)
{
    free(V(*aa));
    free(*aa);
    *aa = NULL;
}
```

# Implementare coada generica – A [15]

---

```
int MutaQ (void *ad, void *as)
{
    if(DIMDIF(as,ad)) return -1;
    if(VIDA(as)) return 0;
    if(PLINA(ad)) return -2;

    memcpy(SC(ad), IC(as), DIME(as));
    Avans(ad, SC(ad));
    Avans (as, IC(as));
    return 1;
}
```



# Implementare coada generica – A [16]

---

```
int ConcatQ(void *ad, void *as)
{
    if(DIMDIF(as,ad)) return -1;
    if(VIDA(as)) return 0;
    if(Liber(ad) < Ocupat(as)) return -2;

    for(; IC(as) != SC(as);
        Avans(ad,SC(ad)),Avans(as,IC(as)))
        memcpy(SC(ad), IC(as), DIME(as));
    return 1;
}
```

# Exemple prelucrari

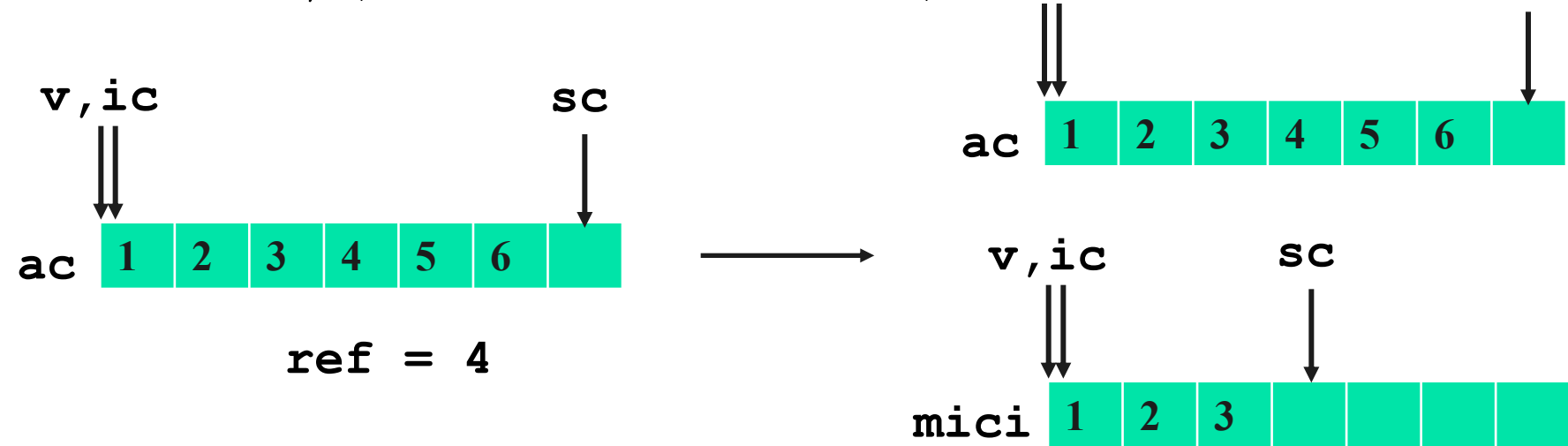
---

```
void* CopieMaiMici(void* ac, int ref)
```

- Fie o coada implementata generic, care contine elemente int
- Sa se construiasca o noua coada in care sunt copiate elementele < ref
- Elementele din cele doua cozi vor pastra ordinea relativa din coada initiala

# Exemple prelucrari

```
void* CopieMaiMici(void* ac, int ref)
{ void *mici, *caux; /* adresa coada rezultat, aux */
  int x; /* valoare element */ v,ic          sc
```



```
/*- initializare cozi de lucru -*/
```

```
mici = InitQ(DimES(ac), MAX);
```

```
if (!mici) return NULL;
```

```
caux = InitQ(DimES(ac), MAX);
```

```
if (!caux){ DistrQ(&mici); return NULL; }
```

```
}
```

# Exemple prelucrari

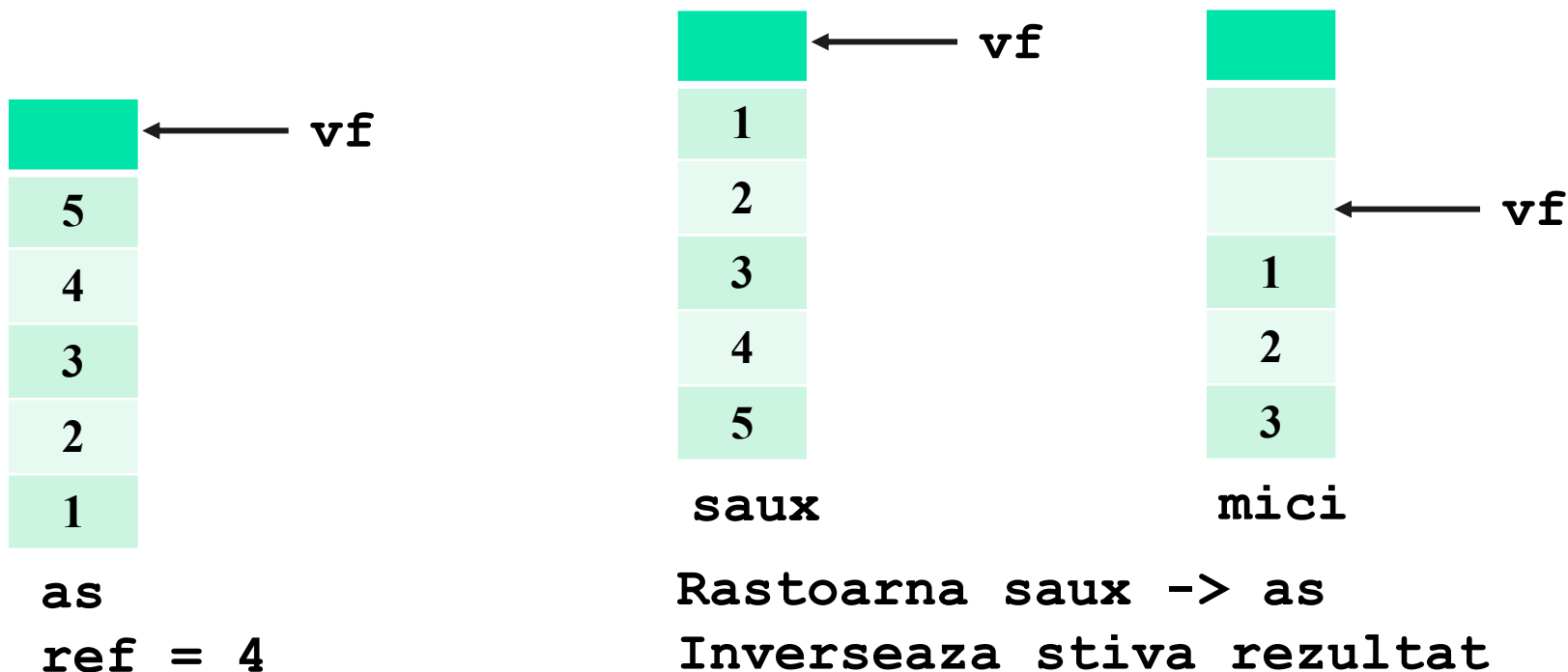
---

```
/*- transfer din coada sursa in cozile de lucru -*/  
while (ExtrQ(ac, &x))  
{ if(x < ref)  
    IntrQ(mici, &x);  
  IntrQ(caux, &x);  
}  
ac = caux; //?? NU SE POATE  
ConcatQ(ac, caux); /* muta elementele in coada initiala*,  
DistrQ(&caux); /* nu mai este necesara */  
  
/* intoarce rezultat */  
return mici;  
}
```

# Exemple prelucrari

```
void* CopieMaiMici(void* as, int ref, )
```

- Fie o stiva implementata generic, care contine elemente int
- Sa se construiasca o noua stiva in care sunt transferate elementele  $< \text{ref}$
- Elementele din cele doua stive vor pastra ordinea relativa din stiva initiala



# Exemple prelucrari

---

```
void* TransferMaiMici(int ref, void* as)
{ void * mici, *saux; /* adrese stive de lucru */
  int x; /* valoare element */
  /*- initializare stive de lucru -*/

  mici = InitS(DimES(as), MAX);
  if (!mici) return NULL;

  saux = InitS(DimES(as), MAX);
  if (!mari){ DistrS(&mici); return NULL; }
}
```

# Exemple prelucrari

---

```
/*- transfer din stiva sursa in stivele de lucru -*/
```

```
while (Pop(as, &x))  
{ if(x < ref)  
    Push(mici, &x);  
    Push(saux, &x);  
}
```

```
/* elementele mari raman in stiva sursa */
```

```
Rastoarna(as, saux);  
DistrS(&saux); /* este vida si nu mai este necesara */
```

```
/* stiva elementelor mici -> rezultat */
```

```
InvS(mici);  
return mici;  
}
```

# Exemple prelucrari

---

```
void* TransferMaiMici(void* ac, int ref)
/* coada */
```

```
while (ExtrQ(ac, &x))
{ if(x < ref)
    IntrQ(mici, &x);
  IntrQ(caux, &x);
}
```

*Devine*

```
while (ExtrQ(ac, &x))
{ if(x < ref)
    IntrQ(mici, &x);
  else
    IntrQ(caux, &x);
}
```



# Exemple prelucrari

```
void* TransferMaiMici(void* ac, int ref)
/* nu se cunoaste tipul elementelor
   din coada /structura */
int x;    → //void* x;

/* alocare x */
x = (int*)malloc(sizeof(int));
//x = malloc(DimEQ(ac));
if(!x) { /*elibereaza structure alocate*/
    return NULL;}

...
while (ExtrQ(ac, x))
{ if(x < ref)
    IntrQ(mici, x);
  else
    IntrQ(caux, x);
}

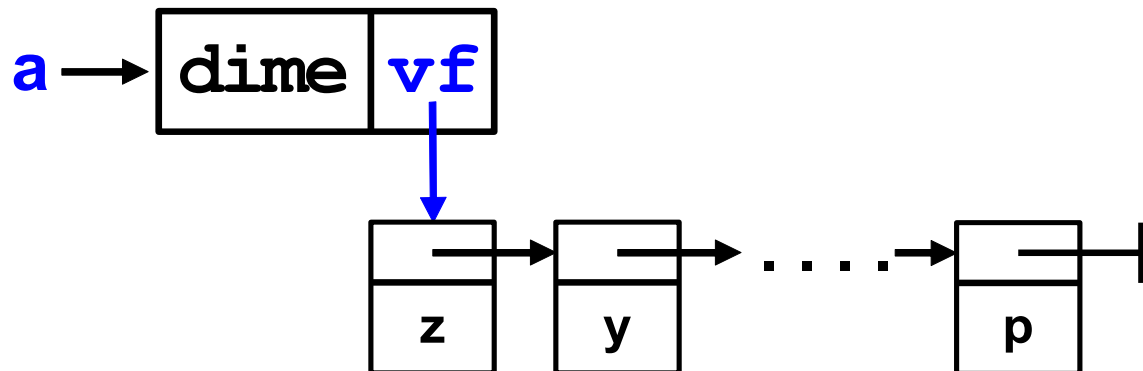
...
free(x);
```

# Implementare **stiva** generica – B [1]

## B. Folosind o **lista**

```
typedef struct celst
{ struct celst *urm; /* adr.urmatoarei celule */
  void* info;        /* adresa informatie */
} TCelSt, *ACelSt;
```

```
typedef struct stiva
{ size_t dime;        /* dim.element */
  ACelSt vf;          /* adresa celulei din varf */
} TStiva, *ASt;
```



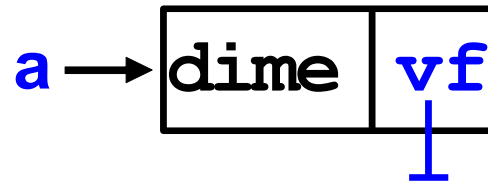
# Implementare **stiva** generica – B [2]

---

```
#define VF(a) ((ASt)(a))->vf  
#define DIME(a) ((ASt)(a))->dime)
```

## Initializare

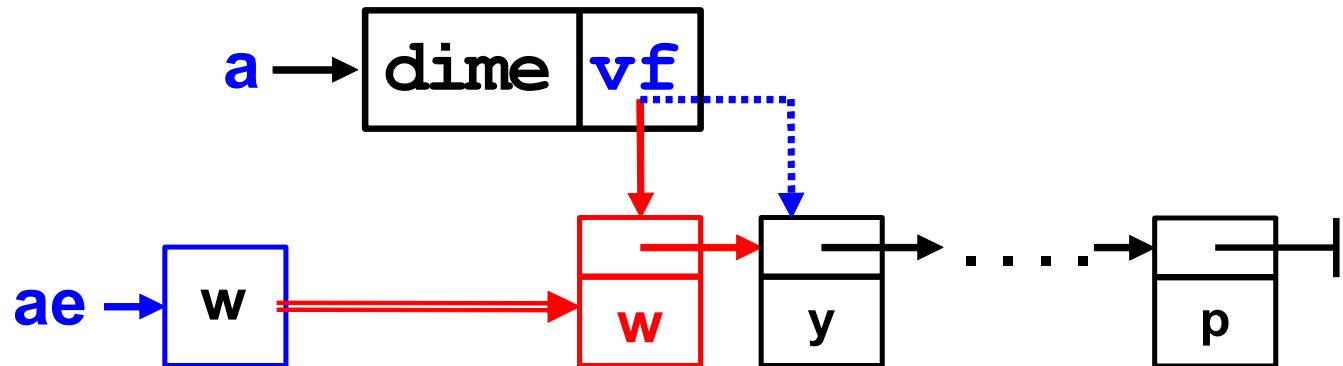
```
void* a = InitS(sizeof(tip_element));  
void* a = InitS(sizeof(int));
```



Stiva vida: **VF(a) == NULL**

# Implementare **stiva** generica – B [3]

## Inserare



```
ACelSt aux = (ACelSt)malloc(sizeof(TCelSt));  
if(!aux) return 0;
```

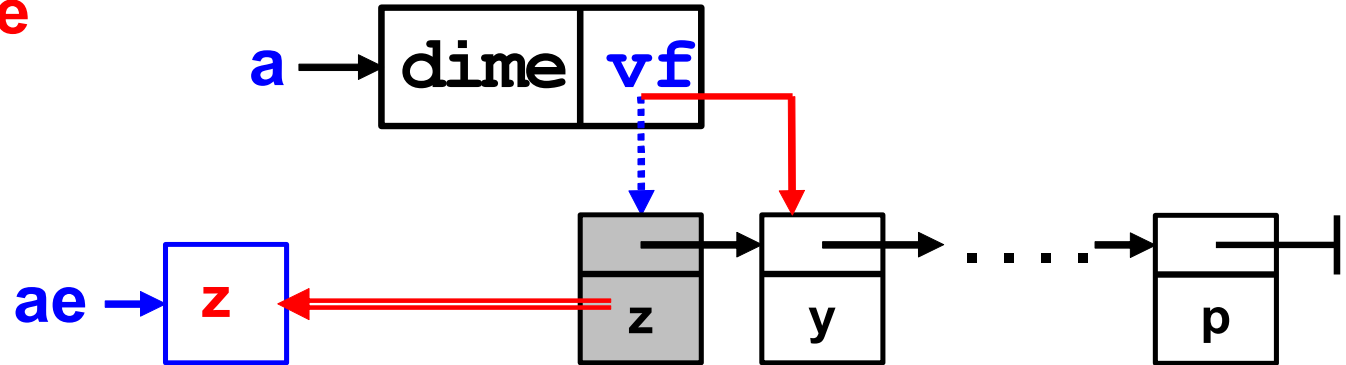
Aloca si testeaza aux->info de dimensiune `((TStiva*)a)->dime`

Copiază în aux->info valoarea de la adresa ae  
`memcpy(aux->info, ae, ((TStiva*)a)->dime);`

```
aux->urm = ((TStiva*)a)->vf;  
((TStiva*)a)->vf = aux;
```

# Implementare **stiva** generica – B [3]

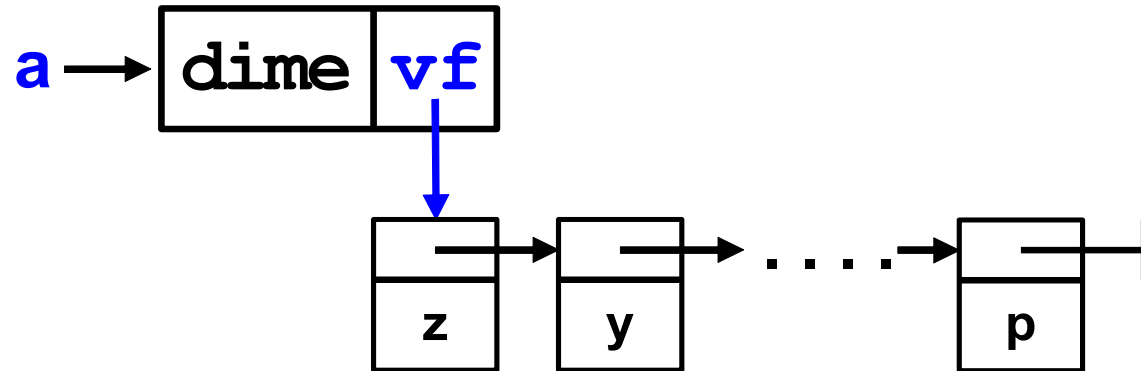
## Extragere



```
int Pop(void* s, void* ae)
{ if stiva vida (((TStiva*)s)->vf == NULL) return 0;
  memcpy(ae, (((TStiva*)s)->vf->info,
            (((TStiva*)s)->dime);
  ACelSt aux = (((TStiva*)s)->vf;
  (((TStiva*)s)->vf = aux->urm;
  free(aux->info);
  free(aux);
  return 1;
}
```

# Implementare **stiva** generica – B [3]

## ResetS



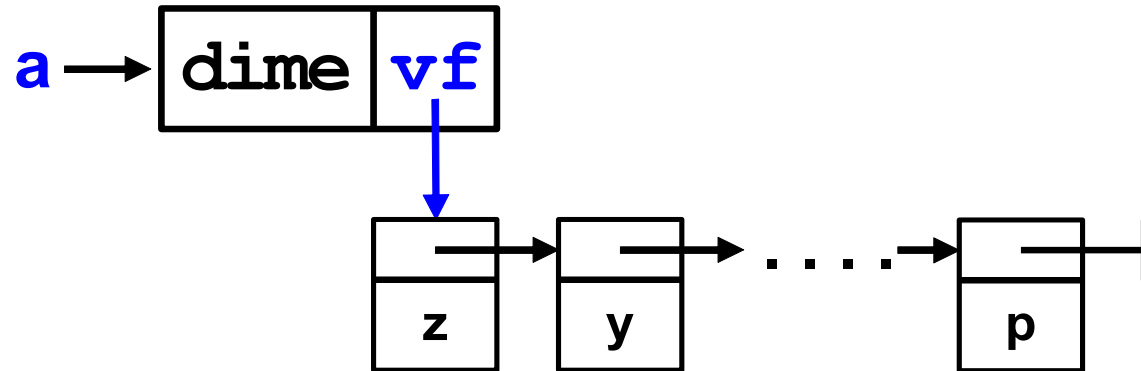
```
l = a->vf (cu cast)
```

```
Distruge lista l (l lista generica)
```

```
a->vf = NULL (cu cast)
```

# Implementare **stiva** generica – B [3]

## DistrS



```
l = a->vf (cu cast)
```

```
Distruge lista l (l lista generica)
```

```
a->vf = NULL (cu cast)
```

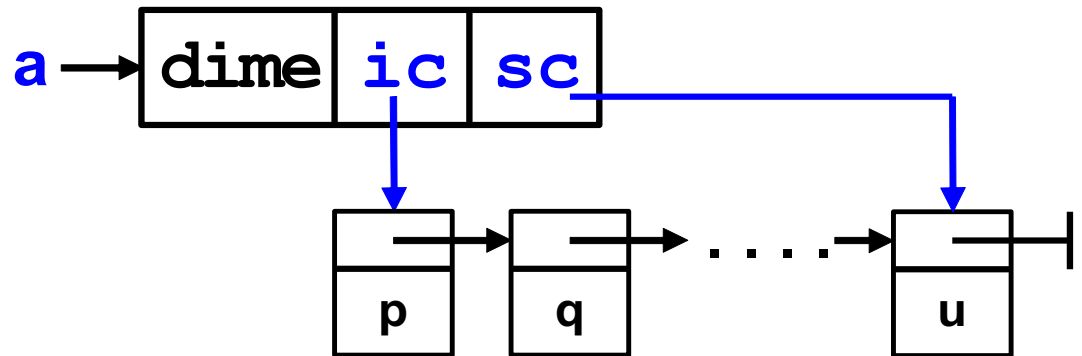
```
free(a)
```

# Implementare coada generica – B [4]

## B. Folosind o **lista** deschisa, fara santinela

```
typedef struct cel      /* dimensiune variabila */
{ struct cel * urm;
  void* info;
} TCell, *ACell;
```

```
typedef struct coada
{ size_t dime;      /* dim.element */
  ACell ic, sc;     /* adr.prima, ultima celula */
} TCoada, *AQ;
```



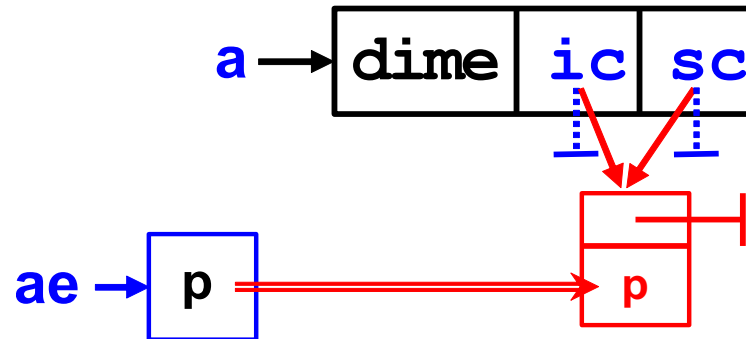


# Implementare coada generica – B [5]

Coada vida:  $a \rightarrow ic == NULL \ \&\& \ a \rightarrow sc == NULL$

## Inserare

- in coada vida



Alocare celula + test alocare

$aux \rightarrow info = ae$  //ae trebuie sa fie alocat

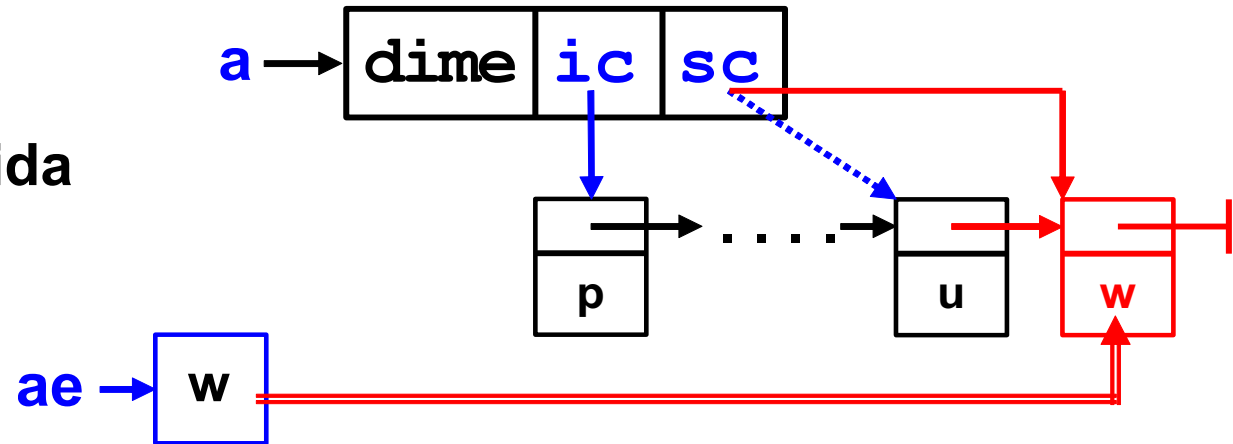
$Aux \rightarrow urm = NULL$

$ic = sc = aux$

# Implementare **coada** generica – B [5]

## Inserare

- in coada nevada



Alocare celula + test alocare

aux->info = ae //ae trebuie sa fie alocat

aux->urm = NULL

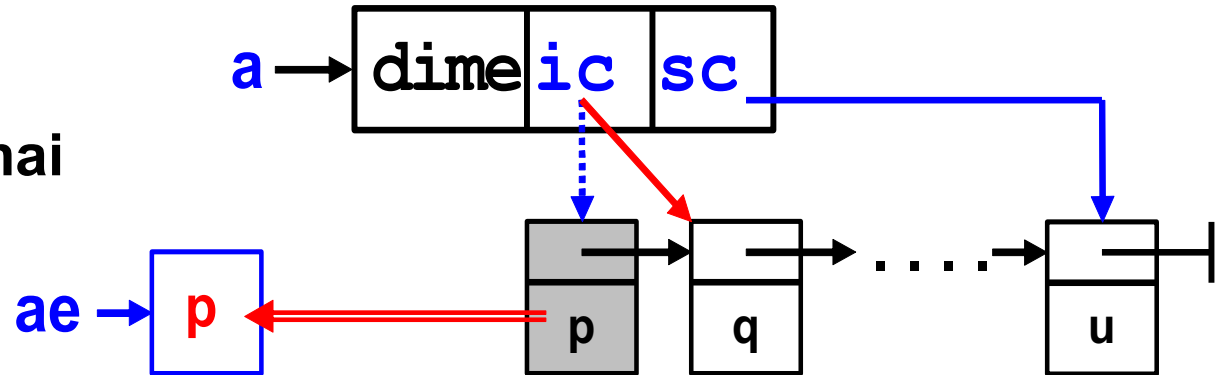
a->sc->urm = aux

a->sc = aux

# Implementare coada generica – B [6]

**Extragere(void\* a, void\* ae)**

- din coada cu mai multe elemente

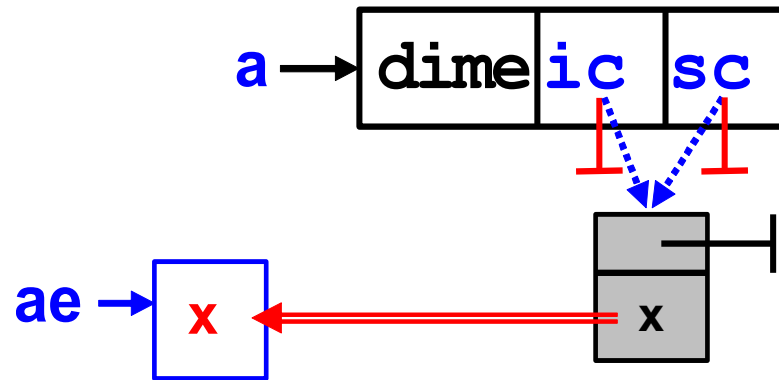


```
aux = a->ic  
a->ic = aux->urm
```

```
memcpy(ae, aux->info, a->dime)  
free(aux->info)  
free(aux)
```

# Implementare **coada** generica – B [6]

**Extragere** - din coada cu un singur element



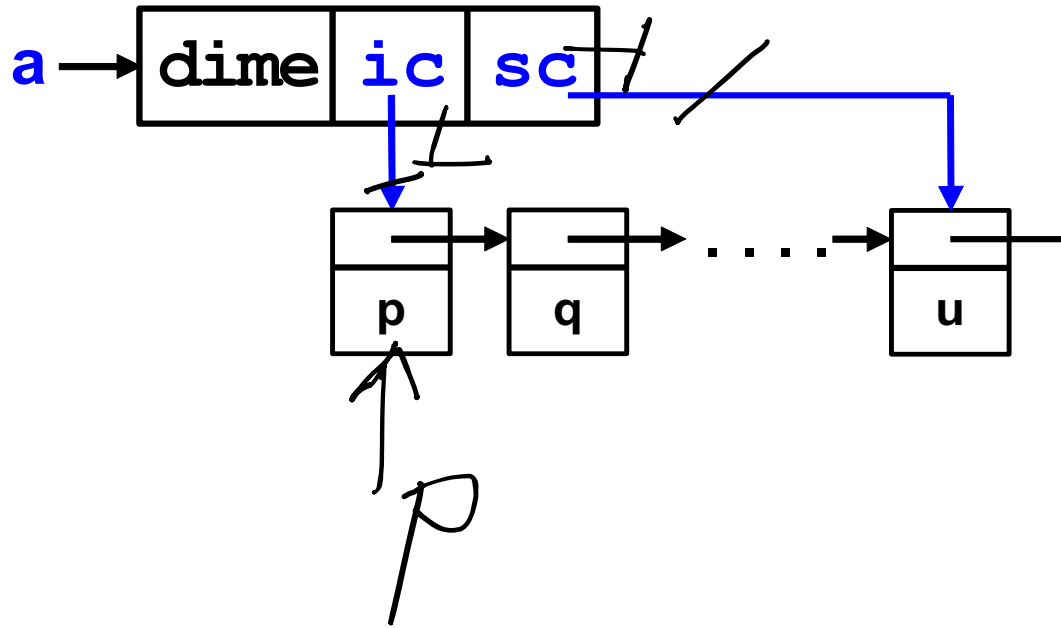
```
aux = a->ic  
a->ic = aux->urm
```

```
memcpy(ae, aux->info, a->dime)  
free(aux->info)  
free(aux)
```

```
if a->ic == NULL a->sc = NULL
```

# Implementare **coada** generica – B [6]

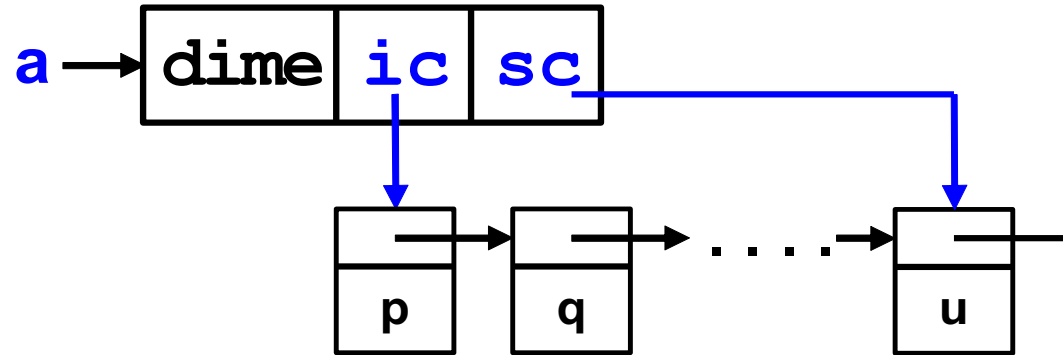
## ResetQ



```
p = a->ic  
a->ic = NULL  
a->sc = NULL  
Distruge lista p
```

# Implementare **coada** generica – B [6]

## **DistrQ**



```
ResetQ(a)  
free(a)
```